

Language Tutorial

In our language tutorial, we chose to make the installation and use of MME quite simple to create ease-of-use for previously non-programmer users. To utilize MME, simply type “git clone https://github.com/daisyapto/CMPSC470_MME” into your terminal. This will clone the entire MME language to your local machine. From here, simply run the “MME.py” Python script, and you will be prompted with an MME terminal. From here, there are three main options users can type into the terminal: an MME code statement, “test mode”, and “exit”. When an MME code statement is ran (with proper syntax), the result is instantly returned. If there are syntax errors, the terminal prints an error message. When “test mode” is entered, the MME language runs an internal test of sample code statements (3 of every type of function that we implemented. When “exit” is entered, the MME language exits its execution.

The ReadMe file in our project repository includes more details on documentation and simple general steps of testing and utilizing MME. The following sample code statements are examples of the proper syntax to use in the MME language. A crucial note is that every symbol requires that there is a space in between them to return the proper, accurate result; this is an essential syntax rule of MME.

The essential MME code reference guide (also used in “test mode”):

1. “Solve for x : $y = x + 1$ ”
2. “Solve for y : $x = (y + 1)^2$ ”
3. “Solve for z : $y = z - 3$ ”
4. “Simplify : $y = x + 1 + 2$ ”
5. “Simplify : $z = y + x + 1 + 3 + 5$ ”

6. "Simplify : $x = y + 1 + 3 + 3 + (5 - 3)^2$ "
7. "Derive for x : $y = x^2 + 1$ "
8. "Derive for y : $x = y^3 + (2 * y) + 1$ "
9. "Derive for z : $y = z^2 + 1$ "
10. "Integrate for x : $y = x^2 + x + 1$ "
11. "Integrate for y : $x = y + y^2 + 1$ "
12. "Integrate for z : $y = z^2 + 1$ "
13. "Average : [3 , 5 , 7]"
14. "Average : [8 , 190 , 3 , 6 , 7 , 3 , 7 , 8]"
15. "Average : [19.5 , 3.5 , 3.2 , 4.8 , 4.4 , 9.99]"
16. "Mode : [3 , 5 , 5 , 7 , 2 , 2 , 2 , 2 , 5 , 6]"
17. "Mode : [3 , 5 , 5 , 7 , 6 , 7 , 7 , 7 , 7 , 3 , 4 , 5]"
18. "Mode : [9 , 9 , 9 , 9 , 9 , 9 , 9 , 10 , 10 , 10 , 14 , 15 , 6]"
19. "Max : [3 , 5 , 5 , 7 , 2 , 2 , 2 , 2 , 5 , 6]"
20. "Max : [3 , 5 , 5 , 7 , 6 , 7 , 7 , 7 , 7 , 3 , 4 , 5]"
21. "Max : [9 , 9 , 9 , 9 , 9 , 9 , 9 , 10 , 10 , 10 , 14 , 15 , 6]"
22. "Min : [3 , 5 , 5 , 7 , 2 , 2 , 2 , 2 , 5 , 6]"
23. "Min : [3 , 5 , 5 , 7 , 6 , 7 , 7 , 7 , 7 , 3 , 4 , 5]"
24. "Min : [9 , 9 , 9 , 9 , 9 , 9 , 9 , 10 , 10 , 10 , 14 , 15 , 6]"

Language Reference Manual

Our language reference manual is in 5 main sections: solving, simplifying, deriving, integrating, and statistical calculating. To solve equations, use “solve for x” (or any other variable), and MME will specifically find the variable that needs to be solve for. To simplify, MME follows a simple methodology for this. The function will use the sympy library’s simplify function to reduce an expression to its simplest form. To derive, the methodology is simple. The input must be an expression, not an equation, and it must include one variable for no syntax errors. To integrate, a similar structure and code statement must be used. For statistical calculations, a list of numerical values must be on the right side of the separation colon, using a Python-like syntax for the list, such as “[3, 5, 7]” to ensure a proper, error-free calculation. Once again, an essential syntax rule of MME for all these calculations is that every symbol has a space between it. This ensures proper tokenization of your code statement. A lack of spaces between your symbols will cause a semantic error, meaning the MME code statement will execute, but it will return inaccurate results.